

DP

PALAKURTHI LALITH PRAKASH

2303A52012

22.08.2026

740. Delete and Earn

```
class Solution {

    int[] dp;
    int[] points;

    public int deleteAndEarn(int[] nums) {

        points = new int[10001];
        for (int num : nums) points[num] += num;
        dp = new int[10001];
        for (int i = 0; i < dp.length; i++) dp[i] = -1;

        return solve(10000);
    }

    int solve(int i) {
        if (i <= 0) return 0;
        if (dp[i] != -1) return dp[i];
        return dp[i] = Math.max(solve(i - 1), points[i] + solve(i - 2));
    }
}
```

416. Partition Equal Subset Sum

```
class Solution {

    public boolean canPartition(int[] nums) {

        int sum=0;
        for(int i:nums) sum+=i;

        if(sum%2!=0) return false;
```

```

        int trgt = sum/2;

        boolean dp[] = new boolean[trgt+1];
        dp[0]= true;

        for(int i:nums){
            for(int j=trgt;j>=i;j--){
                dp[j] |= dp[j-i];
                if(dp[trgt]) return true;
            }
        }
        return dp[trgt];
    }
}

```

494. Target Sum

```

class Solution {

    public int findTargetSumWays(int[] nums, int target) {

        int totalSum = 0;

        for (int x : nums) totalSum += x;

        if (Math.abs(target) > Math.abs(totalSum)) return 0;

        if ((target + totalSum) % 2 != 0) return 0;

        int sum = (target + totalSum) / 2;

        int[][] dp = new int[nums.length][sum + 1];

        for(int i[:dp) Arrays.fill(i,-1);

        return solve(nums.length - 1, sum, nums, dp);
    }

    private int solve(int idx, int sum, int[] nums, int[][] dp) {

        if (idx == 0) {
            if (sum == 0 && nums[0] == 0) return 2;

```

```

        if(sum==0) return 1;
        if (sum == nums[0]) return 1;
        return 0;
    }

    if (dp[idx][sum] != -1)
        return dp[idx][sum];

    int notPick = solve(idx - 1, sum, nums, dp);

    int pick = 0;

    if (nums[idx] <= sum)
        pick = solve(idx - 1, sum - nums[idx], nums, dp);

    return dp[idx][sum] = pick + notPick;
}
}

```

983. Minimum Cost For Tickets

```

class Solution {

    int[] dp;

    public int mincostTickets(int[] days, int[] costs) {
        dp = new int[days.length];

        for (int i = 0; i < dp.length; i++) {
            dp[i] = -1;
        }

        return solve(0, days, costs);
    }

    int solve(int i, int[] days, int[] costs) {
        if (i >= days.length) return 0;

        if (dp[i] != -1) return dp[i];
        int oneDay = costs[0] + solve(i + 1, days, costs);
        int j = i;

        while (j < days.length && days[j] < days[i] + 7) j++;
    }
}

```

```

        int sevenDay = costs[1] + solve(j, days, costs);
        j = i;
        while (j < days.length && days[j] < days[i] + 30) j++;
        int thirtyDay = costs[2] + solve(j, days, costs);

        dp[i] = Math.min(oneDay, Math.min(sevenDay, thirtyDay));
        return dp[i];
    }
}

```

1049. Last Stone Weight II

```

class Solution {
    public int lastStoneWeightII(int[] stones) {

        int sum = 0;

        for (int i : stones) sum += i;

        int trgt = sum/2;
        int[] dp = new int[trgt + 1];
        for (int i : stones)
            for (int j = trgt; j >= i; j--)
                dp[j] = Math.max( dp[j], dp[j - i] + i );

        return sum - 2 * dp[trgt];
    }
}

```